

Session 3: Constructors & Destructors Topics: Default, Parameterized, Copy constructors. Destructor lifecycle. Example: Auto-save user data upon exit.

Tasks

- 1. Create a class called Playlist with a default constructor that sets the playlist name to 'My Favourites' and prints a welcome message when an object is created.**

```
#include <iostream>
#include <string>
using namespace std;

class Playlist {
private:
    string playlistName;

public:
    Playlist() {
        playlistName = "My Favourites";
        cout << "Welcome! Playlist created successfully." << endl;
    }

    void display() {
        cout << "Playlist Name: " << playlistName << endl;
    }
};

int main() {

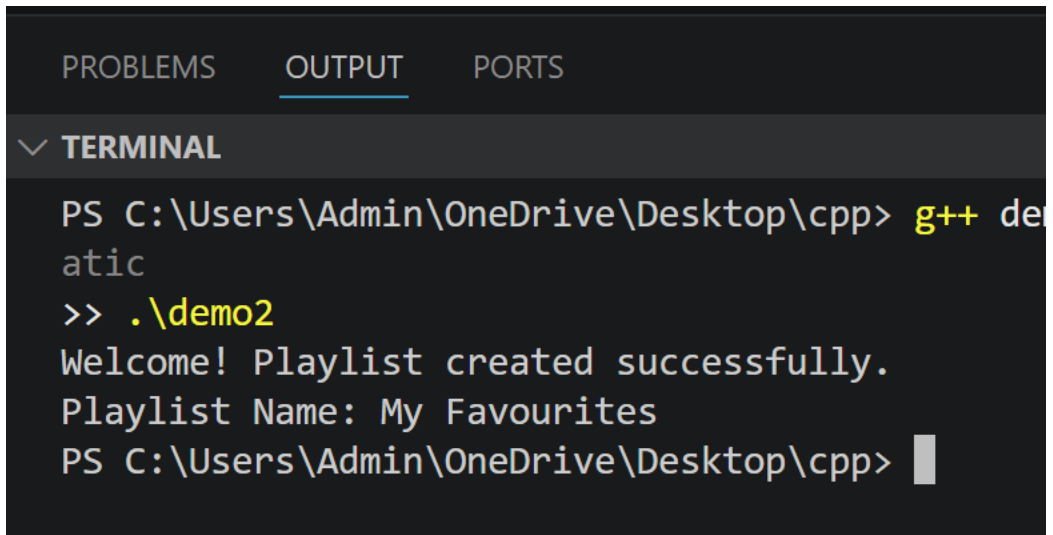
    Playlist p1;

    p1.display();

    return 0;
```

```
}
```

Output-



```
PROBLEMS  OUTPUT  PORTS
✓ TERMINAL
PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ de
atic
>> .\demo2
Welcome! Playlist created successfully.
Playlist Name: My Favourites
PS C:\Users\Admin\OneDrive\Desktop\cpp> |
```

2) Build a class Product for a Flipkart-style app with a parameterized constructor that takes product Name, price, and rating as arguments and displays all details using a displayInfo() method.

Code-

```
#include <iostream>

#include <string>

using namespace std;

class flipkart{
private:
    string name;
    float price;
    float rating;

public:
    flipkart(string n,float p,float r){
        name=n;
        price=p;
        rating=r;
    }

    void displayInfo() {
        cout << "flipkart product name    : " << name << endl;
```

```
        cout << "flipkart product price    : " << price << endl;
        cout << "flipkart product rating  : " << rating<<" / 5" << endl;
    }

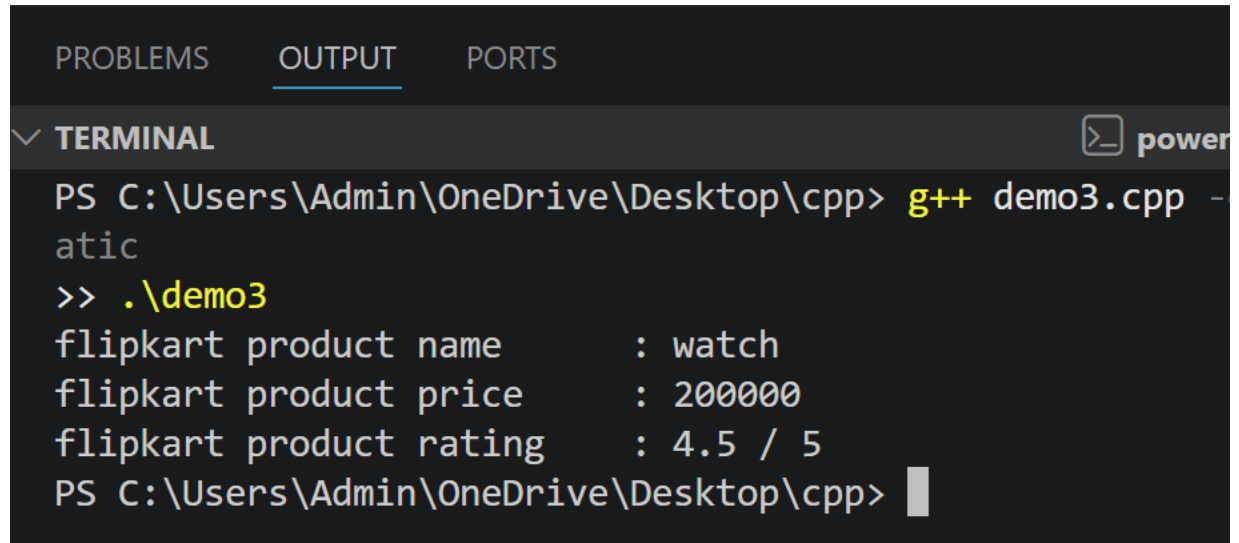
};

int main() {
    flipkart p1("watch",200000,4.5);

    p1.displayInfo();

    return 0;
}
```

Output:



```
PROBLEMS  OUTPUT  PORTS
✓ TERMINAL  power
PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo3.cpp -
atic
>> .\demo3
flipkart product name      : watch
flipkart product price     : 200000
flipkart product rating    : 4.5 / 5
PS C:\Users\Admin\OneDrive\Desktop\cpp>
```

3) Create a class Movie with a parameterized constructor and then use a copy constructor to duplicate a Movie object. Print both the original and copied movie details to show they are identical.

Code-

```
#include <iostream>

#include <string>

using namespace std;

class Movie {

private:

    string movieName;

    int releaseYear;

    float rating;

public:

    Movie(string name, int year, float rate) {

        movieName = name;

        releaseYear = year;

        rating = rate;

    }

    Movie(const Movie &m) {

        movieName = m.movieName;

        releaseYear = m.releaseYear;
```

```
        rating = m.rating;
    }

    void displayInfo() {
        cout << "Movie Name  : " << movieName << endl;
        cout << "Release Year : " << releaseYear << endl;
        cout << "Rating      : " << rating << "/5" << endl;
    }
};

int main() {
    Movie movie1("Pushpa 2", 2024, 4.8);

    Movie movie2(movie1);

    cout << "Original Movie Details:" << endl;
    movie1.displayInfo();

    cout << "\nCopied Movie Details:" << endl;
    movie2.displayInfo();

    return 0;
}
```

Output:

```
PROBLEMS  OUTPUT  PORTS
✓ TERMINAL

Original Movie Details:
Movie Name   : Pushpa 2
Release Year : 2024
Rating       : 4.8/5

Copied Movie Details:
Movie Name   : Pushpa 2
Release Year : 2024
Rating       : 4.8/5
```


**4) Simulate a BookMyShow ticket booking by creating a Ticket class that prints 'Saving your ticket...' in its destructor. Create and delete a Ticket object to demonstrate the destructor lifecycle.

Hint: Use the __del__ method (Python) or ~Ticket() (C++) to define the destructor.**

Code-

```
#include <iostream>

using namespace std;

class Ticket {
public:
    // Constructor
    Ticket() {
        cout << "Ticket booked successfully!" << endl;
    }

    // Destructor
    ~Ticket() {
        cout << "Saving your ticket..." << endl;
    }
};

int main() {

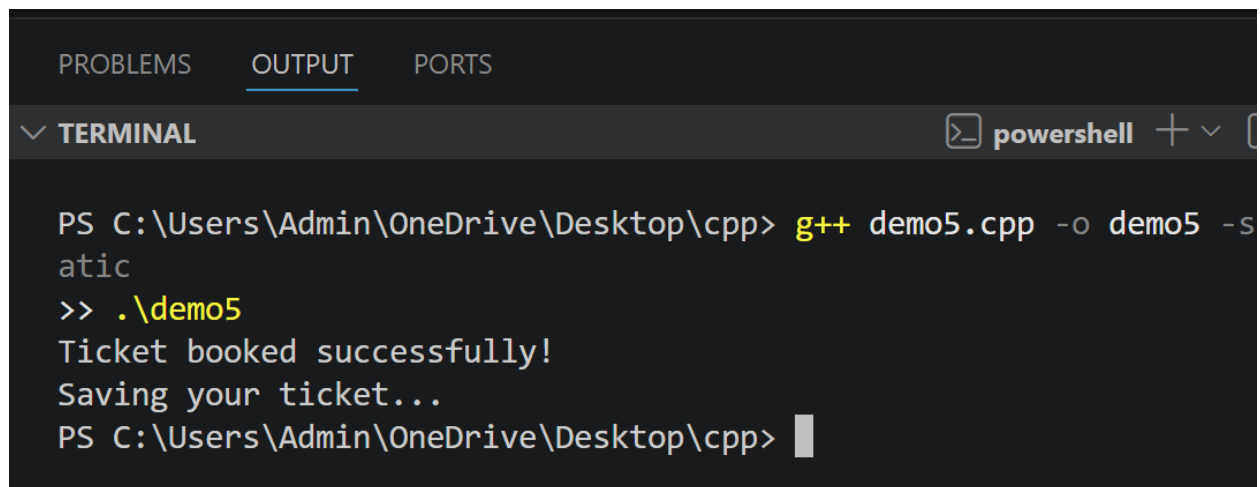
    Ticket *t1 = new Ticket();
```

```
delete t1;
```

```
return 0;
```

```
}
```

Output:



The screenshot shows a Visual Studio IDE window with the 'OUTPUT' tab selected. Below it, the 'TERMINAL' tab is active, displaying a PowerShell session. The user has compiled a C++ file named 'demo5.cpp' using 'g++' with the flags '-o demo5 -std=c++11'. They then executed the resulting binary 'demo5', which printed the messages 'Ticket booked successfully!' and 'Saving your ticket...'. The terminal window title is 'powershell'.

```
PROBLEMS  OUTPUT  PORTS
▼ TERMINAL  powershell + v [
PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo5.cpp -o demo5 -std=c++11
>> .\demo5
Ticket booked successfully!
Saving your ticket...
PS C:\Users\Admin\OneDrive\Desktop\cpp> |
```

**5) Modify your Playlist class so that it auto-saves the playlist name to a file called autosave.txt when the object is destroyed, simulating an auto-save feature like Spotify.

Hint: Write the file-saving code inside the destructor.**

Code-

```
#include <iostream>

#include <fstream>

#include <string>

using namespace std;

class Playlist {

private:

    string playlistName;

public:

    // Parameterized constructor

    Playlist(string name) {

        playlistName = name;

        cout << "Playlist created: " << playlistName << endl;

    }

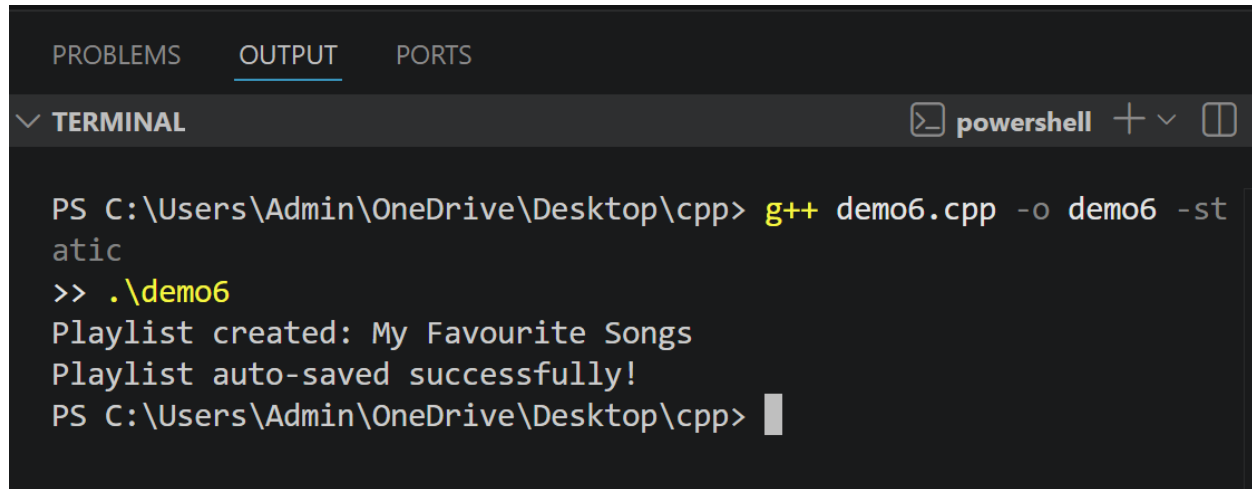
    // Destructor

    ~Playlist() {

        ofstream file("autosave.txt");
```

```
    if (file.is_open()) {  
        file << "Auto-saved Playlist Name: " << playlistName << endl;  
        file.close();  
        cout << "Playlist auto-saved successfully!" << endl;  
    }  
    else {  
        cout << "Unable to open file!" << endl;  
    }  
}  
};  
  
int main() {  
    {  
        Playlist myPlaylist("My Favourite Songs");  
    }  
  
    return 0;  
}
```

Output:



```
PROBLEMS  OUTPUT  PORTS
✓ TERMINAL  powershell + - 
PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo6.cpp -o demo6 -static
>> .\demo6
Playlist created: My Favourite Songs
Playlist auto-saved successfully!
PS C:\Users\Admin\OneDrive\Desktop\cpp>
```